### 5.6.2 Open standards

Building on open standards is a good choice: try to avoid backends that are exclusively accessible via proprietary APIs, or at the very least, have a strategy in mind regarding how you can make these proprietary APIs comply with (industry) standards.

For example, in the context of our observability considerations, support for OpenTelemetry (at least on the road map) is something you want to include in your checklist. Every major cloud provider and ISV already now supports OpenTelemetry—see the respective vendor listing (https://opentelemetry.io/vendors/) on the project home page that details OpenTelemetry support (distributions and OTLP endpoints support).

Not all signal types are equal in this respect:

- Logs tend to have many open standards and formats in use. Standardization is not practical, and hence, standards like OpenTelemetry typically limit themselves to declaring normative mappings from and to established standards, like syslog.
- Metrics are increasingly represented in Prometheus exposition format or its successor, OpenMetrics (which sports exemplars support). Note, however, that there is a lot of old-school stuff, especially from or in non-cloud-native setups, that is still using things like Graphite format, the InfluxDB line protocol, or StatsD.
- Nowadays, it is important to ensure traces are OpenTelemetry compliant.

Next, we will look at a common challenge in the ingestion path.

### 5.6.3 Back pressure

When looking at ingesting signals at scale, you may come across the challenge of dealing with sources that generate, say, log lines or spans at a high rate. To deal with the back pressure—that is, not overloading the backends—you can use a queue between the source, agent, and backend. This allows you to decouple the producer side (signal source or agent) and consumer side (backend).

For example, you could use Apache Kafka (https://kafka.apache.org/) in the egress path of an OpenTelemetry collector configured with the Kafka Exporter (http://mng.bz/N2zN) component in a pipeline, as we discussed in chapter 4. In this configuration, the collector would serve as a short-term, in-memory queue publishing into a Kafka broker, and the backend could, for example, be an S3 bucket that gets populated on a schedule, via a lambda function. Next, we will cover how to manage too many values.

### 5.6.4 Cardinality and queries

In this section, we discuss two related problems when selecting a backend: cardinality and queries. We discussed cardinality explosion in the metrics backend section already. What we mean is that a metric's dimension (e.g., session ID or user ID)